



Laboratório de Programação 3

Aula 11

URLs, views, templates e páginas dinâmicas
no Django

Prof. Anderson Peixoto
Prof^a. Gislene Martins

Revisão

- Na aula anterior, aprendemos a instalar o Django no Linux, criar um projeto, criar um app, configurar URLs e views e exibir uma primeira página no navegador. Esse foi o primeiro contato com a estrutura do framework.

Revisão

- Agora, o próximo passo é compreender melhor como essas partes se conectam e como transformar páginas simples em páginas **dinâmicas**, isto é, páginas que mudam de acordo com os dados enviados pela aplicação.

O fluxo de uma requisição no Django

Toda vez que o usuário acessa uma página no navegador, ocorre um fluxo básico:

1. o navegador envia uma requisição;
2. o Django recebe essa requisição;
3. o sistema consulta as URLs;
4. encontra a view correspondente;
5. a view processa a solicitação;
6. a resposta é enviada ao navegador.

Essa resposta pode ser:

- texto simples;
- uma página HTML;
- dados estruturados.

Diferença entre HttpResponse e render

- Na Aula 1, vimos que uma view pode usar **HttpResponse** para devolver texto diretamente ao navegador. Isso é útil para exemplos iniciais e respostas muito simples.
- Porém, aplicações web reais normalmente exibem páginas HTML. Para isso, o Django usa **render**, que permite carregar um template e mostrá-lo ao usuário.

Assim:

- HttpResponse devolve conteúdo simples;
- render devolve um arquivo HTML, geralmente com dados dinâmicos.

O que é contexto?

- O contexto é o conjunto de dados que a view envia para o template. É ele que permite que o HTML mostre informações vindas do Python.

Exemplos de dados enviados por contexto:

- ✓ nome do curso;
- ✓ nome da escola;
- ✓ nome do professor;
- ✓ listas de disciplinas;
- ✓ informações de turno;
- ✓ mensagens condicionais.

Templates dinâmicos

- Um template dinâmico é um arquivo HTML que usa a sintaxe do Django para exibir dados variáveis.

Essa sintaxe inclui:

Variáveis em templates

Nos templates do Django, usamos `{{ }}` para exibir valores enviados pela view. Isso permite mostrar textos, números e outras informações diretamente no HTML de forma dinâmica.

Templates dinâmicos

Repetição com for

A estrutura `{% for %}` permite percorrer listas enviadas pela view e repetir elementos no template. Esse recurso é muito útil para exibir listas de disciplinas, alunos, produtos, livros e muitos outros dados.

Condicionais com if

A estrutura `{% if %}` permite exibir conteúdos diferentes de acordo com uma condição. Isso torna a página mais inteligente e adaptável ao contexto enviado pela aplicação.

Separação entre lógica e apresentação

Um dos princípios importantes do Django é separar:

- a **lógica** da aplicação, que fica na view;
- a **apresentação**, que fica no template.

Essa separação melhora:

- organização;
- manutenção;
- reaproveitamento;
- clareza do projeto.

Reutilização com template base

- À medida que o projeto cresce, fica ruim repetir o mesmo cabeçalho, rodapé e estrutura em várias páginas. Para resolver isso, usa-se um template base, normalmente chamado `base.html`, do qual outras páginas herdam a estrutura.
- Esse é um primeiro passo importante para aprendermos a organização de interface.

Passo a passo aula 2

Como os computadores do laboratório apagam os arquivos ao reiniciar, no início da aula cada estudante deverá:

- baixar novamente a pasta do projeto;
- abrir a pasta no computador;
- recriar o ambiente virtual;
- reinstalar o Django;
- só então continuar o desenvolvimento.

Etapa 1 — Baixar novamente a pasta do projeto

Cada aluno deve baixar a pasta projeto_escola disponibilizada no Moodle.

Etapa 2 — Abrir o terminal Linux

Abrir o terminal Linux

Passo 1. Abrir o terminal do Linux.

- O terminal continuará sendo a ferramenta principal para executar os comandos do Django.

Etapa 3 — Ir até a pasta onde o projeto foi baixado

Digitar:

cd projeto_escola

Etapa 4 — Verificar se os arquivos do projeto estão presentes

Digitar:

ls

O que esse comando faz

- ls lista os arquivos e pastas do diretório atual.

config core manage.py

Etapa 5 — Criar novamente o ambiente virtual

Digitar:

```
python3 -m venv venv
```

O que faz:

- `python3`: chama o Python 3;
- `-m`: executa um módulo do Python;
- `venv`: módulo responsável por criar ambientes virtuais;
- `venv`: nome da pasta do ambiente virtual.

Etapa 6 — Verificar se o ambiente virtual foi criado

Digitar:

ls

O que deve aparecer:

config core manage.py venv

Etapa 7 — Ativar o ambiente virtual

Digitar:

```
source venv/bin/activate
```

O que esse comando faz

- source: executa um script no terminal atual;
 - venv/bin/activate: é o arquivo que ativa o ambiente virtual.
-
- O terminal geralmente passará a mostrar algo assim no início da linha:
(venv)
 - Quando aparece (venv), isso significa que o ambiente virtual está ativo.
 - A partir desse momento, tudo que instalarmos com pip ficará dentro desse projeto, e não espalhado no sistema inteiro.

Etapa 8 — Instalar o Django novamente

Digitar:

```
pip install django
```

O que esse comando faz

- pip instala pacotes Python;
- install indica que queremos instalar;
- django é o nome do pacote.

Agora estamos baixando e instalando o framework Django dentro do ambiente virtual. Uma biblioteca ou framework normalmente é instalado pelo nome do pacote no pip.

Etapa 9 — Confirmar a instalação do Django

Digitar:

```
python -m django --version
```

O que esse comando faz

- `python -m django` executa o Django como módulo Python;
- `--version` mostra a versão instalada.

Isso confirma que o Django realmente foi instalado e está acessível dentro do ambiente virtual.

Etapa 10 — Rodar o projeto novamente

Digitar:

```
python manage.py runserver
```

O que esse comando faz

- python: executa o interpretador Python;
- manage.py: arquivo que permite rodar comandos administrativos do projeto;
- runserver: comando que inicia o servidor local de desenvolvimento.

O que deve aparecer

Starting development server at <http://127.0.0.1:8000/>

Esse servidor permite testar o projeto no próprio computador/navegador.

Etapa 11 — Abrir o navegador

Abrir o navegador e acessar:

`http://127.0.0.1:8000/`

O que deve aparecer

- A página inicial do projeto deve abrir normalmente.
- Esse teste serve para confirmar que o projeto foi recuperado corretamente e está pronto para ser editado.

Etapa 12 — Parar o servidor para começar as alterações da Aula 2

No terminal, pressionar:

Ctrl + C

O que isso faz

- Interrompe a execução do servidor.

Sempre que o servidor estiver rodando, o terminal ficará “preso” nessa tarefa.
Para voltar a digitar novos comandos, é preciso pará-lo com Ctrl + C.

A partir daqui começa a continuação
normal da Aula 2

Etapa 13 — Abrir o arquivo `core/views.py`

Esse é o arquivo onde ficam as funções que respondem às rotas do sistema.

Etapa 14 — Alterar a página sobre para usar template HTML

Substituir o conteúdo de **core/views.py** por:

```
from django.shortcuts import render

def inicio(request):
    return render(request, 'core/inicio.html')

def sobre(request):
    return render(request, 'core/sobre.html')
```

Etapa 14 — Alterar a página sobre para usar template HTML

Substituir o conteúdo de **core/views.py** por:

```
from django.shortcuts import render //Importa a função render, usada para  
carregar um template HTML.
```

```
def inicio(request):
```

```
    return render(request, 'core/inicio.html') //Renderiza a página inicial.
```

```
def sobre(request):
```

```
    return render(request, 'core/sobre.html') //Agora a página sobre também  
será exibida com HTML, e não mais com texto puro.
```

Antes, a rota **sobre** devolvia apenas um texto. Agora, ela também vai carregar um arquivo HTML, como acontece em aplicações reais.

Etapa 15 — Criar o arquivo sobre.html

core/templates/core/sobre.html

Conteúdo:

```
<!DOCTYPE html>
```

```
<html lang="pt-BR">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Sobre</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Sobre o projeto</h1>
```

```
  <p>Esta página apresenta informações sobre o sistema criado na aula de Django.</p>
```

```
</body>
```

```
</html>
```

Etapa 16 — Criar uma nova view chamada curso

Alterar **core/views.py** para:

```
from django.shortcuts import render
```

```
def inicio(request):
```

```
    return render(request, 'core/inicio.html')
```

```
def sobre(request):
```

```
    return render(request, 'core/sobre.html')
```

```
def curso(request):
```

```
    contexto = {
```

```
        'nome_curso': 'Curso Técnico em Desenvolvimento de Sistemas',
```

```
        'professor': 'Professor Exemplo',
```

```
        'carga_horaria': 1200,
```

```
        'disciplinas': ['Python', 'Banco de Dados', 'Django', 'HTML e CSS'],
```

```
        'turno': 'Noturno'
```

```
    }
```

```
    return render(request, 'core/curso.html', contexto)
```

Etapa 16 — Criar uma nova view chamada curso

```
curso(request):    //Cria uma nova view chamada curso.  
contexto = { ... } //Cria um dicionário com os dados que serão enviados ao template.  
'nome_curso'      //Nome do curso que será mostrado na página.  
'professor'       //Nome do professor.  
'carga_horaria'   //Carga horária do curso.  
'disciplinas'     //Lista de disciplinas.  
'turno'           //Valor usado para testar uma condição no template.  
return render(request, 'core/curso.html', contexto) //Carrega o arquivo curso.html e  
envia o contexto para ele.
```

Aqui começamos a criar páginas dinâmicas. O HTML não será mais fixo, porque ele vai receber dados da view.

Etapa 17 — Criar a nova rota **curso/**

Abrir **core/urls.py** e alterar para:

```
from django.urls import path  
from .views import inicio, sobre, curso
```

```
urlpatterns = [  
    path('', inicio, name='inicio'),  
    path('sobre/', sobre, name='sobre'),  
    path('curso/', curso, name='curso'),  
]
```

Etapa 17 — Criar a nova rota **curso/**

from .views import inicio, sobre, curso *//Agora importamos também a view curso.*

path('curso/', curso, name='curso') *//Cria a rota /curso/, associada à função curso.*

Toda nova página precisa ter uma view e uma rota correspondente. Sem a rota, o navegador não sabe como chegar à função.

Etapa 18 — Criar o template curso.html

Criar o arquivo:

`core/templates/core/curso.html`

Etapa 18 — Criar o template curso.html

Conteúdo:

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8">
    <title>Curso</title>
  </head>
  <body>
    <h1>{{ nome_curso }}</h1>
    <p><strong>Professor:</strong> {{
      professor }}</p>
    <p><strong>Carga horária:</strong> {{
      carga_horaria }} horas</p>
```

```
<h2>Disciplinas</h2>
  <ul>
    {% for disciplina in disciplinas %}
      <li>{{ disciplina }}</li>
    {% endfor %}
  </ul>

  {% if turno == 'Noturno' %}
    <p>Esta turma funciona no período
    noturno.</p>
  {% endif %}
</body>
</html>
```

Etapa 18 — Criar o template curso.html

{{ nome_curso }}

Mostra o valor da variável nome_curso.

{{ professor }}

Mostra o nome do professor.

{{ carga_horaria }}

Mostra a carga horária.

{% for disciplina in disciplinas %}

Percorre a lista disciplinas.

{{ disciplina }}

Mostra cada item da lista.

{% if turno == 'Noturno' %}

Exibe uma mensagem apenas se o valor de turno for Noturno.

Aqui o HTML passa a ser dinâmico: ele mostra dados que vieram da view, repete itens da lista e toma decisões com base em condições.

Etapa 19 — Rodar o servidor novamente

`python manage.py runserver`

Etapa 20 — Testar as páginas no navegador

Abrir:

<http://127.0.0.1:8000/>

<http://127.0.0.1:8000/sobre/>

<http://127.0.0.1:8000/curso/>

Etapa 21 — Parar o servidor novamente

Ctrl + C

Etapa 22 — Criar o template base

Criar o arquivo:

core/templates/core/base.html

Conteúdo:

```
<!DOCTYPE html>
```

```
<html lang="pt-BR">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>{% block title %}Sistema Django{% endblock %}</title>
```

```
</head>
```

```
<body>
```

```
  <header>
```

Etapa 22 — Criar o template base

Conteúdo:

```
<h1>Sistema da Aula de Django</h1>
```

```
<nav>
```

```
<a href="/">Início</a> |
```

```
<a href="/sobre/">Sobre</a> |
```

```
<a href="/curso/">Curso</a>
```

```
</nav>
```

```
<hr>
```

```
</header>
```


Etapa 22 — Criar o template base

Conteúdo:

```
<main>
{% block content %}
{% endblock %}
</main>
<footer>
<hr>
<p>Aula 2 — Django</p>
</footer>
</body>
</html>
```

Etapa 22 — Criar o template base

Esse arquivo servirá como base visual para todas as páginas do projeto.

Etapa 23 — Adaptar o inicio.html

Arquivo:

`core/templates/core/inicio.html`

Novo conteúdo:

Etapa 23 — Adaptar o inicio.html

```
{% extends 'core/base.html' %}
```

```
{% block title %}Página Inicial{% endblock %}
```

```
{% block content %}
```

```
<h2>Bem-vindo ao Django</h2>
```

```
<p>Esta é a primeira página HTML do projeto.</p>
```

```
<h3>Turma</h3>
```

```
<p>Curso Técnico em Desenvolvimento de Sistemas</p>
```

```
<h3>Estudante</h3>
```

```
<p>Nome do aluno</p>
```

```
{% endblock %}
```

Etapa 23 — Adaptar o inicio.html

**Agora a página inicial herda a estrutura do
base.html.**

Etapa 24 — Adaptar o sobre.html

Arquivo:

`core/templates/core/sobre.html`

Novo conteúdo:

Etapa 24 — Adaptar o sobre.html

```
{% extends 'core/base.html' %}
```

```
{% block title %}Sobre{% endblock %}
```

```
{% block content %}
```

```
    <h2>Sobre o projeto</h2>
```

```
    <p>Esta página apresenta informações sobre o sistema criado na aula de  
Django.</p>
```

```
{% endblock %}
```

Etapa 25 — Adaptar o curso.html

Arquivo:

`core/templates/core/curso.html`

Novo conteúdo:

Etapa 25 — Adaptar o curso.html

Novo conteúdo:

```
{% extends 'core/base.html' %}
```

```
{% block title %}Curso{% endblock %}
```

```
{% block content %}
```

```
    <h2>{{ nome_curso }}</h2>
```

```
    <p><strong>Professor:</strong> {{ professor }}</p>
```

```
    <p><strong>Carga horária:</strong> {{ carga_horaria }}  
    horas</p>
```

Etapa 25 — Adaptar o curso.html

Novo conteúdo:

```
<h3>Disciplinas</h3>
```

```
<ul>
```

```
  {% for disciplina in disciplinas %}
```

```
    <li>{{ disciplina }}</li>
```

```
  {% endfor %}
```

```
</ul>
```

```
{% if turno == 'Noturno' %}
```

```
  <p>Esta turma funciona no período noturno.</p>
```

```
{% endif %}
```

```
{% endblock %}
```

Etapa 26 — Rodar o servidor novamente

Novo conteúdo:

```
python manage.py runserver
```

Etapa 27 — Testar novamente no navegador

Abrir:

`http://127.0.0.1:8000/`

`http://127.0.0.1:8000/sobre/`

`http://127.0.0.1:8000/curso/`

O que observar

Agora todas as páginas compartilham a mesma estrutura visual.

Resumo

- Até aqui, nós saímos de páginas simples para páginas dinâmicas. Agora a view não apenas devolve uma resposta, mas também pode enviar dados para o HTML. Isso faz com que a página mostre informações variáveis, vindas do Python.

Laboratório de Programação 3

Aula 11

URLs, views, templates e páginas dinâmicas
no Django

Prof. Anderson Peixoto
Prof^a. Gislene Martins